

EJEMPLO ADAPTER

🔴 Ejemplo sencillo del Patrón Adapter

🔴 Caso de uso:

Supongamos que tenemos una clase existente `EnchufeEuropeo` que solo tiene el método `conectar220V()`, pero en EE.UU. los dispositivos funcionan con 110V.

Queremos usar `EnchufeEuropeo` en EE.UU. sin modificar su código.

```
// 1 Interfaz que el cliente espera usar
interface EnchufeAmericano {
    void conectar110V();
}

// 2 Clase existente (Incompatible con EnchufeAmericano)
class EnchufeEuropeo {
    void conectar220V() {
        System.out.println("Conectado a 220V (Enchufe Europeo)");
    }
}

// 3 Adapter: Convierte 110V en 220V
class Adaptador implements EnchufeAmericano {
    private EnchufeEuropeo enchufeEuropeo;

    public Adaptador(EnchufeEuropeo enchufeEuropeo) {
        this.enchufeEuropeo = enchufeEuropeo;
    }

    @Override
    public void conectar110V() {
        System.out.println("Adaptador convierte 110V a 220V...");
        enchufeEuropeo.conectar220V();
    }
}

// 4 Cliente: Usa el enchufe americano sin preocuparse por la conversión
public class Main {
    public static void main(String[] args) {
        EnchufeEuropeo enchufeEuropeo = new EnchufeEuropeo();
        EnchufeAmericano adaptador = new Adaptador(enchufeEuropeo);

        adaptador.conectar110V(); // Cliente usa 110V, pero el adaptador lo convierte a 220V
    }
}
```

🔴 Ejemplo de Mal Diseño (Sin Adapter)

🔴 Problema:

Tenemos una aplicación de carga de dispositivos que usa un cargador USB-C, pero ahora queremos soportar dispositivos con MicroUSB sin modificar el código del cargador.

🔴 Problemas del diseño sin Adapter:

- **Alto acoplamiento** → El código del cliente necesita conocer **detalles internos** de cada tipo de conector.
- **Dificultad para agregar nuevos conectores** → Cada vez que se necesite un nuevo tipo de conector, se debe modificar el código existente.
- **No es escalable** → Si hay múltiples tipos de cargadores y dispositivos, el código se volverá inmanejable.

Código sin Adapter (MAL DISEÑO)

```
java                                                                    Copy Edit

// 1 Clase que representa un teléfono con MicroUSB
class TelefonoMicroUSB {
    public void cargarMicroUSB() {
        System.out.println("Cargando teléfono con MicroUSB...");
    }
}

// 2 Clase que representa un cargador USB-C
class CargadorUSBC {
    public void cargarUSBC() {
        System.out.println("Cargando dispositivo con USB-C...");
    }
}

// 3 Cliente que intenta cargar un teléfono MicroUSB con un cargador USB-C (mal diseño)
public class Main {
    public static void main(String[] args) {
        TelefonoMicroUSB telefono = new TelefonoMicroUSB();
        CargadorUSBC cargador = new CargadorUSBC();

        // X Intentamos cargar un teléfono MicroUSB con un cargador USB-C, lo que no es posible
        System.out.println("Intentando cargar el teléfono MicroUSB con un cargador USB-C...")
        telefono.cargarMicroUSB(); // No es compatible
        cargador.cargarUSBC(); // No es compatible

        // X El cliente tiene que manejar manualmente la incompatibilidad
        System.out.println("Necesitamos modificar el código para que funcione...");
    }
}
```

Código con Adapter (BUEN DISEÑO)

```
java                                                                    Copy Edit

// 1 Interfaz común para todos los cargadores
interface Cargador {
    void cargar();
}

// 2 Clase concreta que representa un cargador USB-C
class CargadorUSBC implements Cargador {
    @Override
    public void cargar() {
        System.out.println("Cargando dispositivo con USB-C...");
    }
}
```

```
// 3 Clase que representa un teléfono con MicroUSB (NO compatible con CargadorUSBC)
class TelefonoMicroUSB {
    public void cargarMicroUSB() {
        System.out.println("Cargando teléfono con MicroUSB...");
    }
}

// 4 Adapter: Convierte un cargador USB-C en un cargador compatible con MicroUSB
class AdaptadorMicroUSB implements Cargador {
    private TelefonoMicroUSB telefonoMicroUSB;

    public AdaptadorMicroUSB(TelefonoMicroUSB telefonoMicroUSB) {
        this.telefonoMicroUSB = telefonoMicroUSB;
    }

    @Override
    public void cargar() {
        System.out.println("Adaptador en uso: convirtiendo USB-C a MicroUSB...");
        telefonoMicroUSB.cargarMicroUSB(); // Delegamos la carga al teléfono MicroUSB
    }
}
```

```
// 5 Cliente: Usa el Adapter sin preocuparse por la compatibilidad
public class Main {
    public static void main(String[] args) {
        TelefonoMicroUSB telefono = new TelefonoMicroUSB();
        Cargador adaptador = new AdaptadorMicroUSB(telefono);

        System.out.println("Usando un adaptador para cargar el teléfono MicroUSB con un cargador USB-C");
        adaptador.cargar(); // ✅ Ahora el teléfono MicroUSB se carga correctamente usando el adaptador
    }
}
```

EJEMPLO BRIDGE

📌 Ejemplo de Mal Diseño (Sin Bridge)

📌 Problema:

Supongamos que tenemos un sistema donde hay diferentes dispositivos (TV y Radio) y diferentes tipos de controles remotos (Básico y Avanzado).

📌 Problema del diseño sin Bridge:

- Alta cantidad de clases innecesarias → Para cada combinación de dispositivo y control remoto, necesitamos una subclase (ej. `ControlBasicoTV`, `ControlBasicoRadio`, `ControlAvanzadoTV`, `ControlAvanzadoRadio` ...).
- Código difícil de escalar → Si agregamos un nuevo dispositivo (ej. `Proyector`), tenemos que crear más clases.
- Fuerte acoplamiento → Las clases están atadas a un tipo específico de dispositivo.

Código sin Bridge (MAL DISEÑO)

java

 Copy  Edit

```
// 1 Dispositivo TV con un control básico
class ControlBasicoTV {
    public void encender() {
        System.out.println("Encendiendo TV con control básico...");
    }

    public void apagar() {
        System.out.println("Apagando TV con control básico...");
    }
}

// 2 Dispositivo Radio con un control básico
class ControlBasicoRadio {
    public void encender() {
        System.out.println("Encendiendo Radio con control básico...");
    }
}
```

```
    public void apagar() {
        System.out.println("Apagando Radio con control básico...");
    }
}

// 3 Dispositivo TV con un control avanzado
class ControlAvanzadoTV {
    public void encender() {
        System.out.println("Encendiendo TV con control avanzado...");
    }

    public void apagar() {
        System.out.println("Apagando TV con control avanzado...");
    }

    public void silenciar() {
        System.out.println("Silenciando TV...");
    }
}
```

```
// 4 Dispositivo Radio con un control avanzado
class ControlAvanzadoRadio {
    public void encender() {
        System.out.println("Encendiendo Radio con control avanzado...");
    }

    public void apagar() {
        System.out.println("Apagando Radio con control avanzado...");
    }

    public void silenciar() {
        System.out.println("Silenciando Radio...");
    }
}
```

```
// 5 Cliente con un alto acoplamiento
public class Main {
    public static void main(String[] args) {
        ControlBasicoTV tvBasico = new ControlBasicoTV();
        tvBasico.encender();
        tvBasico.apagar();

        ControlAvanzadoRadio radioAvanzado = new ControlAvanzadoRadio();
        radioAvanzado.encender();
        radioAvanzado.silenciar();
        radioAvanzado.apagar();
    }
}
```

🔴 Problemas del código

- ✗ Alta cantidad de clases → Para cada tipo de control y dispositivo, tenemos una nueva subclase.
- ✗ Difícil de mantener → Si agregamos un nuevo dispositivo, como un **Proyector**, hay que crear más clases.
- ✗ Código repetitivo → Muchas clases tienen código similar, pero en diferentes combinaciones.

✅ Solución con el Patrón Bridge

- 🔴 Separaremos la lógica del control remoto y los dispositivos, de modo que cualquier control pueda operar cualquier dispositivo sin necesidad de múltiples clases.

📄 Código con Bridge (BUEN DISEÑO)

```
// 1 Implementación base para dispositivos
interface Dispositivo {
    void encender();
    void apagar();
}

// 2 Implementaciones concretas de dispositivos
class TV implements Dispositivo {
    public void encender() { System.out.println("TV encendida"); }
    public void apagar() { System.out.println("TV apagada"); }
}

class Radio implements Dispositivo {
    public void encender() { System.out.println("Radio encendida"); }
    public void apagar() { System.out.println("Radio apagada"); }
}
```

```
// 3 Abstracción: Control Remoto (usa un Dispositivo)
abstract class ControlRemoto {
    protected Dispositivo dispositivo;

    public ControlRemoto(Dispositivo dispositivo) {
        this.dispositivo = dispositivo;
    }

    abstract void encender();
    abstract void apagar();
}
```

```
// 4 Implementaciones concretas de controles
class ControlBasico extends ControlRemoto {
    public ControlBasico(Dispositivo dispositivo) { super(dispositivo); }

    public void encender() {
        System.out.println("Usando control básico:");
        dispositivo.encender();
    }

    public void apagar() {
        System.out.println("Usando control básico:");
        dispositivo.apagar();
    }
}
```

```
class ControlAvanzado extends ControlRemoto {
    public ControlAvanzado(Dispositivo dispositivo) { super(dispositivo); }

    public void encender() {
        System.out.println("Usando control avanzado con temporizador:");
        dispositivo.encender();
    }

    public void apagar() {
        System.out.println("Usando control avanzado con temporizador:");
        dispositivo.apagar();
    }
}
```

```
// 5 Cliente: Puede usar cualquier control con cualquier dispositivo
public class Main {
    public static void main(String[] args) {
        Dispositivo tv = new TV();
        Dispositivo radio = new Radio();

        ControlRemoto controlBasico = new ControlBasico(tv);
        ControlRemoto controlAvanzado = new ControlAvanzado(radio);

        controlBasico.encender(); // "Usando control básico: TV encendida"
        controlBasico.apagar();   // "Usando control básico: TV apagada"

        controlAvanzado.encender(); // "Usando control avanzado con temporizador: Radio encendida"
        controlAvanzado.apagar();   // "Usando control avanzado con temporizador: Radio apagada"
    }
}
```

EJEMPLO COMPOSITE

📌 Ejemplo de Mal Diseño (Sin Composite)

📌 Problema:

Supongamos que estamos desarrollando un sistema de gestión de archivos y carpetas.

Queremos que el usuario pueda manejar archivos individuales y carpetas que contienen otros archivos o carpetas de la misma manera.

📌 Problema del diseño sin Composite:

- El código diferencia manualmente entre archivos y carpetas → Se necesitan `if-else` en el código del cliente.
- Dificultad para manejar estructuras anidadas → Una carpeta puede contener otras carpetas, y el código se vuelve más complejo.
- Alto acoplamiento → Si queremos agregar un nuevo tipo de `Elemento`, hay que modificar el código en varios lugares.

📄 Código sin Composite (MAL DISEÑO)

```
java
import java.util.ArrayList;
import java.util.List;

// 1 Clase Archivo (Elemento individual)
class Archivo {
    private String nombre;

    public Archivo(String nombre) {
        this.nombre = nombre;
    }

    public void mostrar() {
        System.out.println("Archivo: " + nombre);
    }
}
```



```
// 2 Clase Carpeta (Elemento compuesto, pero sin una estructura común con
class Carpeta {
    private String nombre;
    private List<Object> elementos = new ArrayList<>(); // X Se usa Object, lo que es mala p

    public Carpeta(String nombre) {
        this.nombre = nombre;
    }

    public void agregar(Object elemento) {
        elementos.add(elemento);
    }

    public void mostrar() {
        System.out.println("Carpeta: " + nombre);
        for (Object elemento : elementos) {
            if (elemento instanceof Archivo) {
                ((Archivo) elemento).mostrar();
            } else if (elemento instanceof Carpeta) {
                ((Carpeta) elemento).mostrar();
            }
        }
    }
}
```

```
// 3 Cliente: Debe diferenciar manualmente entre archivos y carpetas (MAL diseño)
public class Main {
    public static void main(String[] args) {
        Archivo archivo1 = new Archivo("Documento.pdf");
        Archivo archivo2 = new Archivo("Imagen.jpg");

        Carpeta carpeta1 = new Carpeta("Mis Archivos");
        carpeta1.agregar(archivo1);
        carpeta1.agregar(archivo2);

        Carpeta carpeta2 = new Carpeta("Proyectos");
        carpeta2.agregar(new Archivo("Proyecto1.docx"));
        carpeta2.agregar(carpeta1); // Agrega otra carpeta

        carpeta2.mostrar(); // X El cliente debe manejar manualmente las diferencias entre a
    }
}
```

🔴 Problemas del código

- ✗ El cliente tiene que verificar manualmente si es un `Archivo` o una `Carpeta` usando `instanceof`.
- ✗ Dificultad para manejar estructuras jerárquicas → No hay una interfaz común entre `Archivo` y `Carpeta`.
- ✗ Fuerte acoplamiento → Si queremos agregar otro tipo de `Elemento`, hay que modificar varias partes del código.

✓ Solución con el Patrón Composite

📌 Definiremos una interfaz común `ElementoSistema` para archivos y carpetas, permitiendo que ambos se manejen de manera uniforme.

📄 Código con Composite (BUEN DISEÑO)

```
// 1 Interfaz común para archivos y carpetas
interface ElementoSistema {
    void mostrar();
}

// 2 Clase Archivo (Hoja)
class Archivo implements ElementoSistema {
    private String nombre;

    public Archivo(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public void mostrar() {
        System.out.println("Archivo: " + nombre);
    }
}
```

```
// 3 Clase Carpeta (Composite)
class Carpeta implements ElementoSistema {
    private String nombre;
    private List<ElementoSistema> elementos = new ArrayList<>();

    public Carpeta(String nombre) {
        this.nombre = nombre;
    }

    public void agregar(ElementoSistema elemento) {
        elementos.add(elemento);
    }

    @Override
    public void mostrar() {
        System.out.println("Carpeta: " + nombre);
        for (ElementoSistema elemento : elementos) {
            elemento.mostrar();
        }
    }
}
```

📄 Copy ✎ Edit

```
// 4 Cliente: Usa archivos y carpetas sin preocuparse por su estructura interna
public class Main {
    public static void main(String[] args) {
        Archivo archivo1 = new Archivo("Documento.pdf");
        Archivo archivo2 = new Archivo("Imagen.jpg");

        Carpeta carpeta1 = new Carpeta("Mis Archivos");
        carpeta1.agregar(archivo1);
        carpeta1.agregar(archivo2);

        Carpeta carpeta2 = new Carpeta("Proyectos");
        carpeta2.agregar(new Archivo("Proyecto1.docx"));
        carpeta2.agregar(carpeta1); // Agrega otra carpeta

        carpeta2.mostrar();
    }
}
```

🚩 Salida esperada

makefile

📄 Copy 🗑 Edit

```
Carpeta: Proyectos
Archivo: Proyecto1.docx
Carpeta: Mis Archivos
Archivo: Documento.pdf
Archivo: Imagen.jpg
```

◆ Explicación

- ✓ **ElementoSistema** (interfaz común) → Tanto **Archivo** como **Carpeta** implementan **mostrar()**, permitiendo una estructura uniforme.
- ✓ **Archivo** representa **elementos individuales** en la estructura.
- ✓ **Carpeta** puede contener **archivos y otras carpetas**, permitiendo **estructuras jerárquicas**.
- ✓ El cliente usa **ElementoSistema** sin preocuparse por si está trabajando con un archivo o una carpeta.

EJEMPLO DECORATOR

🚩 Ejemplo de Mal Diseño (Sin Decorator)

🚩 Problema:

Estamos desarrollando un sistema de cafetería, donde los clientes pueden pedir un café básico y agregarle extras como leche o azúcar.

🚩 Problema del diseño sin Decorator:

- **Alta cantidad de clases innecesarias** → Para cada combinación de ingredientes, hay que crear una nueva clase (**CafeConLeche**, **CafeConAzucar**, **CafeConLecheYAzucar** ...).
- **Dificultad para agregar nuevos ingredientes** → Si agregamos **Chocolate**, **Canela**, etc., debemos seguir creando clases combinadas.
- **Poca flexibilidad** → No podemos modificar los ingredientes en tiempo de ejecución.

Código sin Decorator (MAL DISEÑO)

```
java Copy Edit

// 1 Clase base para un café simple
class Cafe {
    public String getDescripcion() {
        return "Café simple";
    }

    public double getCosto() {
        return 5.0;
    }
}
```

```
// 2 Subclase con Leche (cada extra necesita una nueva subclase)
class CafeConLeche extends Cafe {
    @Override
    public String getDescripcion() {
        return super.getDescripcion() + " con leche";
    }

    @Override
    public double getCosto() {
        return super.getCosto() + 1.5;
    }
}
```

```
// 3 Subclase con azúcar
class CafeConAzucar extends Cafe {
    @Override
    public String getDescripcion() {
        return super.getDescripcion() + " con azúcar";
    }

    @Override
    public double getCosto() {
        return super.getCosto() + 0.5;
    }
}
```

```
// 4 Subclase con Leche y azúcar (Combinaciones crecen exponencialmente) Copy Edit
class CafeConLecheYAzucar extends Cafe {
    @Override
    public String getDescripcion() {
        return super.getDescripcion() + " con leche y azúcar";
    }

    @Override
    public double getCosto() {
        return super.getCosto() + 2.0;
    }
}

// 5 Cliente con un diseño inflexible
public class Main {
    public static void main(String[] args) {
        Cafe cafe1 = new CafeConLeche();
        System.out.println(cafe1.getDescripcion() + " cuesta $" + cafe1.getCosto());

        Cafe cafe2 = new CafeConLecheYAzucar();
        System.out.println(cafe2.getDescripcion() + " cuesta $" + cafe2.getCosto());
    }
}
```

🔴 Problemas del código

- ❌ Crecimiento exponencial de clases → Cada nueva combinación requiere una nueva subclase.
- ❌ Poca flexibilidad → Si queremos agregar ingredientes dinámicamente, no podemos.
- ❌ Dificultad para escalar → Si agregamos más ingredientes (Chocolate, Canela), debemos crear aún más clases.

✅ Solución con el Patrón Decorator

- 🔴 Usaremos Decorator para agregar ingredientes dinámicamente a un café sin necesidad de crear múltiples subclases.

📄 Código con Decorator (BUEN DISEÑO)

```
// 1 Interfaz base para el café
interface Cafe {
    String getDescripcion();
    double getCosto();
}

// 2 Implementación básica del café
class CafeSimple implements Cafe {
    public String getDescripcion() {
        return "Café simple";
    }

    public double getCosto() {
        return 5.0; // Precio base
    }
}
```

```
// 3 Decorador base
abstract class CafeDecorator implements Cafe {
    protected Cafe cafe;

    public CafeDecorator(Cafe cafe) {
        this.cafe = cafe;
    }

    public String getDescripcion() {
        return cafe.getDescripcion();
    }

    public double getCosto() {
        return cafe.getCosto();
    }
}
```

```
// 4 Decoradores concretos
class Leche extends CafeDecorador {
    public Leche(Cafe cafe) {
        super(cafe);
    }

    public String getDescripcion() {
        return super.getDescripcion() + " con leche";
    }

    public double getCosto() {
        return super.getCosto() + 1.5;
    }
}
```

```
class Azucar extends CafeDecorador {
    public Azucar(Cafe cafe) {
        super(cafe);
    }

    public String getDescripcion() {
        return super.getDescripcion() + " con azúcar";
    }

    public double getCosto() {
        return super.getCosto() + 0.5;
    }
}
```

```
// 5 Cliente: Puede combinar los decoradores como quiera
public class Main {
    public static void main(String[] args) {
        Cafe miCafe = new CafeSimple();
        System.out.println(miCafe.getDescripcion() + " cuesta $" + miCafe.getCosto());

        miCafe = new Leche(miCafe); // Agregar Leche
        System.out.println(miCafe.getDescripcion() + " cuesta $" + miCafe.getCosto());

        miCafe = new Azucar(miCafe); // Agregar azúcar
        System.out.println(miCafe.getDescripcion() + " cuesta $" + miCafe.getCosto());
    }
}
```

EJEMPLO FACADE



Código sin Facade (MAL DISEÑO)

```
class Televisor {
    void encender() { System.out.println("Televisor encendido"); }
    void seleccionarEntrada() { System.out.println("Entrada HDMI seleccionada"); }
}

class Altavoces {
    void encender() { System.out.println("Altavoces encendidos"); }
    void ajustarVolumen(int nivel) { System.out.println("Volumen ajustado a " + nivel); }
}

class ReproductorVideo {
    void encender() { System.out.println("Reproductor de video encendido"); }
    void reproducir(String pelicula) { System.out.println("Reproduciendo: " + pelicula); }
}
```

```
public class Main {
    public static void main(String[] args) {
        // El cliente debe conocer todos los detalles internos 🤯
        Televisor tv = new Televisor();
        Altavoces sonido = new Altavoces();
        ReproductorVideo reproductor = new ReproductorVideo();

        System.out.println("Preparando el sistema para ver una película...");
        tv.encender();
        tv.seleccionarEntrada();
        sonido.encender();
        sonido.ajustarVolumen(10);
        reproductor.encender();
        reproductor.reproducir("El Padrino");
        System.out.println("¡Disfruta tu película!");
    }
}
```



Código Mejorado con Facade

```
// 1 Subsistemas individuales
class Televisor {
    void encender() { System.out.println("Televisor encendido"); }
    void seleccionarEntrada() { System.out.println("Entrada HDMI seleccionada"); }
}

class Altavoces {
    void encender() { System.out.println("Altavoces encendidos"); }
    void ajustarVolumen(int nivel) { System.out.println("Volumen ajustado a " + nivel); }
}

class ReproductorVideo {
    void encender() { System.out.println("Reproductor de video encendido"); }
    void reproducir(String pelicula) { System.out.println("Reproduciendo: " + pelicula); }
}
```

```
// 2 Facade: Simplifica la interacción con los subsistemas
```

```
class HomeTheaterFacade {  
    private Televisor tv;  
    private Altavoces sonido;  
    private ReproductorVideo reproductor;  
  
    public HomeTheaterFacade() {  
        this.tv = new Televisor();  
        this.sonido = new Altavoces();  
        this.reproductor = new ReproductorVideo();  
    }  
  
    public void verPelicula(String pelicula) {  
        System.out.println("Preparando el sistema para ver una película...");  
        tv.encender();  
        tv.seleccionarEntrada();  
        sonido.encender();  
        sonido.ajustarVolumen(10);
```

```
        reproductor.encender();  
        reproductor.reproducir(pelicula);  
        System.out.println("¡Disfruta tu película!");  
    }  
}
```

```
// 3 Cliente: Usa la fachada sin preocuparse por los subsistemas
```

```
public class Main {  
    public static void main(String[] args) {  
        HomeTheaterFacade homeTheater = new HomeTheaterFacade();  
        homeTheater.verPelicula("El Padrino"); // Simplifica la interacción con los sistemas  
    }  
}
```


EJEMPLO FLYWEIGHT

📌 Ejemplo de Mal Diseño (Sin Flyweight)

📌 Problema:

Estamos desarrollando un sistema de dibujo de círculos, donde debemos crear muchos círculos en diferentes posiciones y colores.

📌 Problema del diseño sin Flyweight:

- Cada círculo es un objeto independiente, incluso si tienen el mismo color.
- Uso excesivo de memoria → Si tenemos miles de círculos, creamos miles de objetos redundantes.
- Baja eficiencia → La creación de objetos innecesarios puede afectar el rendimiento.

📄 Código sin Flyweight (MAL DISEÑO)

```
java                                                                    Copy Edit

import java.util.ArrayList;
import java.util.List;

// 1 Clase que representa un círculo (Cada instancia es única)
class Circulo {
    private String color;
    private int x, y;

    public Circulo(String color, int x, int y) {
        this.color = color;
        this.x = x;
        this.y = y;
    }

    public void dibujar() {
        System.out.println("Dibujando círculo de color " + color + " en (" + x + ", " + y + ")");
    }
}
```

```
// 2 Cliente que crea múltiples círculos sin reutilización (ineficiente)
public class Main {
    public static void main(String[] args) {
        List<Circulo> circulos = new ArrayList<>();

        // X Creamos círculos duplicados, sin reutilizar los de mismo color
        circulos.add(new Circulo("Rojo", 10, 20));
        circulos.add(new Circulo("Azul", 30, 40));
        circulos.add(new Circulo("Rojo", 50, 60)); // X Nuevo objeto en vez de reutilizar el
        circulos.add(new Circulo("Azul", 70, 80)); // X Nuevo objeto en vez de reutilizar el

        for (Circulo c : circulos) {
            c.dibujar();
        }
    }
}
```

🔴 Problemas del código

- ✗ Cada círculo se crea como un objeto independiente, incluso si tienen el mismo color.
- ✗ Uso innecesario de memoria → Si queremos 1000 círculos, estamos creando 1000 objetos en lugar de reutilizarlos.
- ✗ Baja eficiencia → Demasiadas instancias pueden impactar el rendimiento.

✅ Solución con el Patrón Flyweight

- 🔴 Usaremos una fábrica para reutilizar círculos con el mismo color y reducir el uso de memoria.

📄 Código con Flyweight (BUEN DISEÑO)

```
// 1 Interfaz común para las figuras
interface Figura {
    void dibujar(int x, int y); // La posición es única
}

// 2 Clase concreta con atributos compartidos (color) y únicos (posición)
class Circulo implements Figura {
    private String color; // Compartido entre múltiples instancias

    public Circulo(String color) {
        this.color = color;
    }

    @Override
    public void dibujar(int x, int y) {
        System.out.println("Dibujando círculo de color " + color + " en (" + x + "," + y + ")");
    }
}
```

```
// 3 Fábrica que reutiliza objetos en función del color
class CirculoFactory {
    private static final Map<String, Circulo> circulos = new HashMap<>();

    public static Circulo obtenerCirculo(String color) {
        if (!circulos.containsKey(color)) {
            circulos.put(color, new Circulo(color)); // Crea solo si no existe
            System.out.println("Creando nuevo círculo de color: " + color);
        }
        return circulos.get(color);
    }
}
```

```
// 4 Cliente: Usa la fábrica para crear círculos sin repetir instancias innecesarias
public class Main {
    public static void main(String[] args) {
        Figura c1 = CirculoFactory.obtenerCirculo("Rojo");
        c1.dibujar(10, 20);

        Figura c2 = CirculoFactory.obtenerCirculo("Azul");
        c2.dibujar(30, 40);

        Figura c3 = CirculoFactory.obtenerCirculo("Rojo"); // Reutiliza el primer círculo rojo
        c3.dibujar(50, 60);
    }
}
```

EJEMPLO PROXY

🚩 Ejemplo de Mal Diseño (Sin Proxy)

🚩 Problema:

Supongamos que tenemos una **base de datos con acceso restringido**, y queremos **controlar el acceso** para que solo ciertos usuarios puedan consultar la información.

🚩 Problema del diseño sin Proxy:

- **No hay control de acceso** → Cualquier usuario puede acceder a la base de datos sin validación.
- **Carga innecesaria de la base de datos** → Se crea una instancia incluso si el usuario no tiene permiso.
- **Poca seguridad** → No hay mecanismos de autenticación antes de acceder a los datos.

📄 Código sin Proxy (MAL DISEÑO)

```
java Copy Edit

// 1 Clase que representa la base de datos real (sin restricciones)
class BaseDeDatos {
    public void consultarDatos() {
        System.out.println("Accediendo a la base de datos...");
        System.out.println("Datos obtenidos: [Usuario: David, Saldo: 5000€]");
    }
}

// 2 Cliente: Puede acceder directamente sin control de acceso (Problema)
public class Main {
    public static void main(String[] args) {
        System.out.println("Usuario admin intentando acceder...");
        BaseDeDatos bd1 = new BaseDeDatos(); // ✗ Instancia creada sin verificar permisos
        bd1.consultarDatos();

        System.out.println("\nUsuario invitado intentando acceder...");
        BaseDeDatos bd2 = new BaseDeDatos(); // ✗ Instancia creada sin restricciones
        bd2.consultarDatos();
    }
}
```

🚩 Problemas del código

- ✗ Cualquier usuario puede acceder a la base de datos, sin validación.
- ✗ Instancia creada aunque el usuario no tenga permisos, desperdiciando recursos.
- ✗ Baja seguridad, no hay autenticación previa al acceso.

✅ Solución con el Patrón Proxy

- 🚩 Usaremos un Proxy para validar el acceso antes de permitir consultas a la base de datos.

📄 Código con Proxy (BUEN DISEÑO)

```
// 1 Interfaz común para la base de datos
interface BaseDeDatos {
    void consultarDatos();
}

// 2 Clase real que accede a los datos (costoso o restringido)
class BaseDeDatosReal implements BaseDeDatos {
    @Override
    public void consultarDatos() {
        System.out.println("Accediendo a la base de datos...");
        System.out.println("Datos obtenidos: [Usuario: David, Saldo: 5000€]");
    }
}

// 3 Proxy: Controla el acceso a la base de datos
class ProxyBaseDeDatos implements BaseDeDatos {
    private BaseDeDatosReal bdReal;
    private String usuario;
```

```
    public ProxyBaseDeDatos(String usuario) {
        this.usuario = usuario;
    }

    @Override
    public void consultarDatos() {
        if (verificarAcceso()) {
            if (bdReal == null) {
                bdReal = new BaseDeDatosReal(); // Carga diferida
            }
            bdReal.consultarDatos();
        } else {
            System.out.println("Acceso denegado: " + usuario + " no tiene permisos.");
        }
    }

    private boolean verificarAcceso() {
        return usuario.equals("admin"); // Solo "admin" tiene acceso
    }
}
```

```
// 4 Cliente: Usa la base de datos a través del proxy
public class Main {
    public static void main(String[] args) {
        BaseDeDatos bd1 = new ProxyBaseDeDatos("admin"); // Usuario con acceso
        bd1.consultarDatos();

        BaseDeDatos bd2 = new ProxyBaseDeDatos("invitado"); // Usuario sin acceso
        bd2.consultarDatos();
    }
}
```